ASSIGNMENT – 3

C++ AND OBJECT ORIENTED PROGRAMMING

Register Number: 192524300

Student Name: Lakshmi Devi J.

GitHub Repository Link: https://github.com/laxmijaya0812-prog/DSA0119--C/tree/main

Assignment Title

Implementation and Evaluation of C++ Constructors, Destructors and Operator Overloading

Q1 – Constructors and Constructor Overloading

Marks: 6

1. Aim

To design and implement a Book class using default, parameterized, overloaded and delegated constructors, while validating the number of copies and understanding constructor initialization.

2. Problem Description

A library creates Book objects in different situations:

- A completely new book with title and number of copies.
- A donated book where only the title is available.
- An empty book record that will be filled later.
- A book must not contain an invalid negative number of copies.

The assignment requires member initializer lists and constructor delegation.

3. Concepts Used

Default Constructor

A constructor that can be called without arguments.

Parameterized Constructor

A constructor that receives values while creating an object.

Constructor Overloading

Multiple constructors having different parameter lists.

Constructor Delegation

One constructor calls another constructor of the same class to avoid duplicate initialization code.

Member Initializer List

Used to initialize data members before the constructor body executes.

4. C++ Program

```
#include <iostream>
#include <string>
using namespace std;
class Book
{
private:
   string title;
   int copies;
   int validCopies(int c)
   {
      return (c >= 0) ? c : 0;
   }
public:
```

```
    // Default constructor
    Book() : Book("", 0)
    {
    }
    // Constructor with title
    Book(string t) : Book(t, 0)
    {
    }
    // Parameterized constructor
    Book(string t, int c)
        : title(t), copies(validCopies(c))
    {
    }
    // Display book details
    void display() const
    {
        cout << "Title   : " << title << endl;
        cout << "Copies  : " << copies << endl;
        cout << endl;
    }
};
int main()
{
```

```
    Book b1;
    Book b2("C++ Programming");
    Book b3("Object Oriented Programming", 5);
    Book b4("Invalid Book", -10);
    cout << "Book 1:" << endl;
    b1.display();
    cout << "Book 2:" << endl;
    b2.display();
    cout << "Book 3:" << endl;
    b3.display();
    cout << "Book 4:" << endl;
    b4.display();
    return 0;
}
```

Output

```
Book 1:
Title   :
Copies  : 0

Book 2:
Title   : C++ Programming
Copies  : 0

Book 3:
Title   : Object Oriented Programming
Copies  : 5

Book 4:
Title   : Invalid Book
Copies  : 0
```

5. Expected Output

Book 1:

Title :

Copies : 0

Book 2:

Title : C++ Programming

Copies : 0

Book 3:

Title : Object Oriented Programming

Copies : 5

Book 4:

Title : Invalid Book

Copies : 0

The negative value -10 is converted to 0, preventing an invalid number of copies.

6. Why Member Initializer Lists Are Preferred

A member initializer list initializes data members directly.

```
Book(string t, int c)
    : title(t), copies(validCopies(c))
{
}
```

This is preferred because:

1. Members are initialized before the constructor body.
2. It avoids unnecessary default initialization followed by assignment.
3. It is required for certain members such as const members and references.
4. It clearly shows how each member is initialized.

7. Initialization Order

Members are initialized in the **order in which they are declared inside the class**, not the order written in the initializer list.

For example:

```
class Example
{
   int a;
   int b;

public:
   Example() : b(20), a(10)
   {
   }
};
```

Even though b appears first in the initializer list, a is initialized first because a is declared first.

8. When Does the Compiler Supply a Default Constructor?

If no constructor is declared by the programmer, the compiler can provide a default constructor.

Example:

```
class Test
{
public:
   int x;
};
```

However, if a parameterized constructor is declared:

```
class Test
{
public:
   Test(int value)
   {
      x = value;
   }
private:
   int x;
};
```

the compiler does not automatically provide a no-argument constructor.

Therefore:

```
Test t;
```

will not compile unless a suitable default constructor is provided.

9. State of Three Objects

After construction:

b1

+----------------------+

| title = "" |

| copies = 0 |

+----------------------+

b2

+----------------------+

| title = C++ |

| copies = 0 |

+----------------------+

b3

+-----------------------------+

| title = Object Oriented |

| copies = 5 |

+-----------------------------+

10. Experimental Investigation

Removing the Default Constructor

If the default constructor is removed and we write:

Book books[3];

the program will fail because the compiler needs a no-argument constructor to create every array element.

Negative Copies

Input:

Book b("Test", -10);

Result:

Copies : 0

Thus, invalid negative copies are prevented.

11. explicit Constructor

A single-argument constructor should often be marked explicit when implicit conversion is undesirable.

Without explicit:

```
class Book
{
public:
   Book(string title)
   {
   }
};


Book b = "C++";
```

An implicit conversion may be allowed through the constructor.

With:

```
explicit Book(string title)
{
}
```

the implicit conversion is prevented.

Instead, the programmer must explicitly construct the object:

```
Book b("C++");
```

12. Alternative Concept – Default Arguments

Instead of multiple overloaded constructors, one constructor could use default arguments:

```
Book(string t = "", int c = 0)
```

```
    : title(t), copies(c >= 0 ? c : 0)
{
}
```

Comparison

Design	**Advantage**
Multiple constructors	Clear meaning for different object-creation situations
Default arguments	Less code
Delegating constructors	Avoid repeated initialization
Validation in one constructor	Easier maintenance

Constructor object creation takes:

Time: O(1)

Space: O(1)

13. Result

The Book class successfully demonstrates default, parameterized and overloaded constructors, constructor delegation, member initializer lists and validation of invalid copy counts.

Q2 – Copy Constructors: Shallow and Deep Copies

Marks: 7

1. Aim

To understand shallow and deep copying of dynamically allocated memory and implement the Rule of Three using a copy constructor, copy assignment operator and destructor.

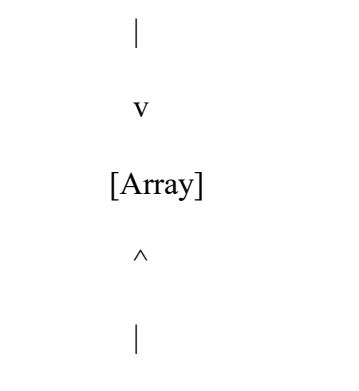
2. Problem Description

A Prescription object stores medicine names in a dynamically allocated array. A shallow copy would cause two objects to share the same memory, which can lead to unwanted changes and double deletion.

The assignment specifically requires independent copies and testing with AddressSanitizer.

3. Shallow Copy

A shallow copy copies the pointer value.

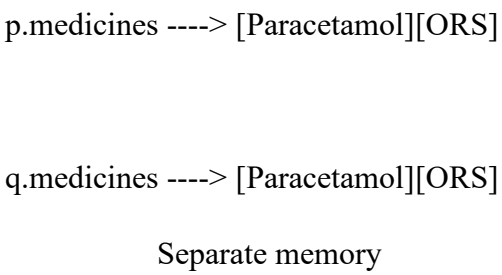


Both objects point to the same array.

Changing q can therefore change p.

4. Deep Copy

A deep copy creates a new array.



The objects become independent.

5. Complete Program

```
#include <iostream>
#include <string>
class Prescription
{
    int n;
    std::string* medicines;
public:
    explicit Prescription(int count)
        : n(count), medicines(new std::string[count])
    {
    }
    // TODO 1: Copy constructor
    Prescription(const Prescription& other)
        : n(other.n), medicines(new std::string[other.n])
    {
        for (int i = 0; i < n; i++)
            medicines[i] = other.medicines[i];
    }
    // TODO 2: Copy assignment
    Prescription& operator=(const Prescription& other)
    {
        if (this != &other)
```

```
    {
        std::string* newMedicines =
            new std::string[other.n];
        for (int i = 0; i < other.n; i++)
            newMedicines[i] = other.medicines[i];
        delete[] medicines;
        medicines = newMedicines;
        n = other.n;
    }
    return *this;
}
// TODO 3: Destructor
~Prescription()
{
    delete[] medicines;
}
void set(int i, const std::string& m)
{
    medicines[i] = m;
}
std::string get(int i) const
{
    return medicines[i];
}
```

```
};
int main()
{
    Prescription p(2);
    p.set(0, "Paracetamol");
    p.set(1, "ORS");
    Prescription q = p;
    q.set(1, "Cetirizine");
    Prescription r(1);
    r = p;
    p.set(0, "Ibuprofen");
    std::cout << p.get(0) << ' ' << p.get(1) << " | " << q.get(0) << ' '<< q.get(1) << " | "<<
r.get(0) << ' ' << r.get(1) << '\n';
    return 0;
}
```

```
Output
Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS
```

6. Expected Output

Ibuprofen ORS | Paracetamol Cetirizine | Paracetamol ORS

This matches the expected output given in the assignment.

7. Explanation

Copy Constructor

Prescription(const Prescription& other)

Creates a new object from an existing object.

A new array is allocated:

medicines(new std::string[other.n])

Then every medicine is copied.

Copy Assignment

p = q;

uses the assignment operator.

The code:

1. Checks self-assignment.
2. Creates new memory.
3. Copies the medicines.
4. Deletes old memory.
5. Updates the pointer.
6. Returns *this.

Destructor

```
~Prescription()
{
   delete[] medicines;
}
```

releases the dynamically allocated array.

8. Rule of Three

The Rule of Three states that if a class manages a resource and needs one of these:

1. Destructor
2. Copy constructor
3. Copy assignment operator

it generally needs all three.

9. When Is the Copy Constructor Called?

Situation 1 – Object Initialization

Prescription q = p;

Situation 2 – Passing by Value

void display(Prescription p)

{

}

A copy can be created for the parameter.

Situation 3 – Returning by Value

A function returning a Prescription may involve copying, although modern C++ can eliminate many such copies through copy elision.

10. Self Assignment

p = p;

The condition:

if (this != &other)

prevents unnecessary deletion and copying from the same object.

11. Alternative Concept – std::vector

Using:

std::vector<std::string>

would automatically manage memory.

Example:

```
class Prescription
{
    std::vector<std::string> medicines;
};
```

The compiler-generated copy operations provide independent vector contents.

This follows the **Rule of Zero** approach.

12. Complexity

For n medicines:

Operation	**Time**	**Space**
Shallow copy	O(1)	O(1)
Deep copy	O(n)	O(n)
Copy assignment	O(n)	O(n)
Destructor	O(n)	O(1)

13. Result

The Prescription class successfully performs deep copying and prevents shared-memory problems and double deletion.

Q3 – Dynamic Constructors and Destructors

Marks: 7

1. Aim

To dynamically allocate a two-dimensional auditorium seat map using a constructor, release the memory using a destructor, and understand object lifetime and RAII.

2. Problem Description

The number of auditorium rows and seats per row varies between events. Therefore, the seat map must be dynamically allocated.

The assignment requires objects on the stack, heap and inside a block, together with booking/cancellation and destructor testing.

3. Dynamic Constructor

A dynamic constructor allocates memory during object construction.

seats = new char*[rows];

Each row is then allocated separately.

4. C++ Program

```
#include <iostream>

using namespace std;

class Event

{

private:

    int rows;

    int seatsPerRow;

    char** seats;

    int booked;

public:


    Event(int r, int s)
```

```
    : rows(r), seatsPerRow(s), booked(0)
{
    cout << "Constructor: Event created" << endl;
    if (rows > 0 && seatsPerRow > 0)
    {
        seats = new char*[rows];
        for (int i = 0; i < rows; i++)
        {
            seats[i] = new char[seatsPerRow];
            for (int j = 0; j < seatsPerRow; j++)
                seats[i][j] = 'O';
        }
    }
    else
    {
        seats = nullptr;
    }
}
void book(int r, int s)
{
    if (r >= 0 && r < rows &&
        s >= 0 && s < seatsPerRow &&
        seats != nullptr &&
        seats[r][s] == 'O')
```

```
    {
        seats[r][s] = 'X';
        booked++;
    }
}
void cancel(int r, int s)
{
    if (r >= 0 && r < rows &&
        s >= 0 && s < seatsPerRow &&
        seats != nullptr &&
        seats[r][s] == 'X')
    {
        seats[r][s] = 'O';
        booked--;
    }
}
void display() const
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < seatsPerRow; j++)
            cout << seats[i][j] << ' ';

        cout << endl;
```

```
        }
    }
    ~Event()
    {
        cout << "Destructor: Event closed. "
             << "Booked seats = " << booked << endl;
        if (seats != nullptr)
        {
            for (int i = 0; i < rows; i++)
                delete[] seats[i];
            delete[] seats;
        }
    }
};
int main()
{
    cout << "Stack object:" << endl;
    Event e1(2, 3);
    e1.book(0, 0);
    e1.book(1, 1);
    e1.display();
    cout << "\nBlock object:" << endl;
    {
        Event e2(1, 2);
```

```
        e2.book(0, 1);
    }
    cout << "\nHeap object:" << endl;
    Event* e3 = new Event(2, 2);
    e3->book(0, 0);
    e3->book(1, 1);
    delete e3;
    cout << "\nProgram completed." << endl;
    return 0;
}
```

```
Output

Stack object:
Constructor: Event created
X O O
O X O

Block object:
Constructor: Event created
Destructor: Event closed. Booked seats = 1

Heap object:
Constructor: Event created
Destructor: Event closed. Booked seats = 2

Program completed.
Destructor: Event closed. Booked seats = 2
```

5. Expected Output

Stack object:

Constructor: Event created

X O O

O X O

Block object:

Constructor: Event created

Destructor: Event closed. Booked seats = 1

Heap object:

Constructor: Event created

Destructor: Event closed. Booked seats = 2

Program completed.

Destructor: Event closed. Booked seats = 2

6. Object Lifetime

Local Object

Event e1(2, 3);

The constructor runs when the object is created and the destructor runs automatically when it leaves its scope.

Block-Scope Object

```
{
    Event e2(1, 2);
}
```

The destructor runs immediately when the block ends.

Heap Object

Event* e3 = new Event(2, 2);

The destructor does not run until:

delete e3;

is executed.

7. What Happens If delete Is Forgotten?

If:

Event* e3 = new Event(2, 2);

is created but never deleted, dynamically allocated memory remains allocated until the program terminates.

This causes a **memory leak**.

8. delete vs delete[]

For:

new char[10]

use:

delete[] ptr;

For:

new Event

use:

delete ptr;

Using the wrong form results in undefined behavior.

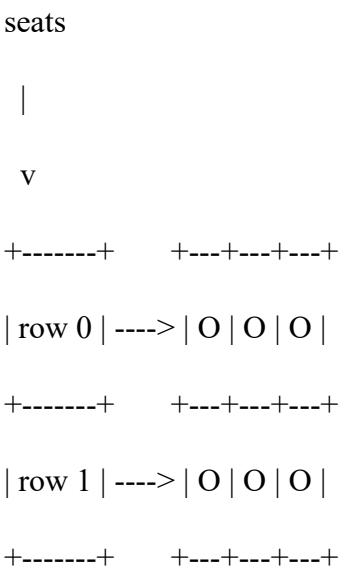
9. Why Destructor Has No Arguments

A destructor is automatically called when an object is destroyed. Therefore it does not require parameters.

A class can have only one destructor, so destructors cannot be overloaded.

10. Memory Layout

For a 2 × 3 seat map:



The first allocation stores row pointers, and each row points to a separately allocated array.

11. RAII

RAII means **Resource Acquisition Is Initialization**.

The resource is acquired during construction and released during destruction.

This makes memory management safer because destruction happens automatically for stack objects.

12. Alternative Design

Instead of allocating every row separately, one continuous block can be used:

char* seats = new char[rows * seatsPerRow];

Access:

seats[r * seatsPerRow + s]

Comparison

Design	Allocations	Access
Row pointers	rows + 1	O(1)
One block	1	O(1)
Vector of vectors	Managed automatically	O(1)

A single block reduces the number of allocations.

13. Complexity

For r rows and s seats:

Allocation: O(r × s)

Destruction: O(r × s)

Seat access: O(1)

Space: O(r × s)

14. Result

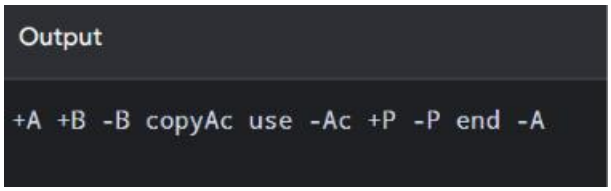
The Event class successfully demonstrates dynamic constructors, destructors, stack objects, block-scope objects, heap objects, seat booking and cancellation.

Q3 – Predict the Output

1. Program

```
#include <iostream>
#include <string>
struct Logger
{
   std::string n;
   Logger(const std::string& s)
      : n(s)
   {
      std::cout << '+' << n << ' ';
   }
   Logger(const Logger& o)
      : n(o.n + "c")
   {
      std::cout << "copy" << n << ' ';
   }
   ~Logger()
   {
      std::cout << '-' << n << ' ';
   }
};
void use(Logger)
{
   std::cout << "use ";
```

```
}
int main()
{
    Logger a("A");
    {
        Logger b("B");
    }
    use(a);
    Logger* p = new Logger("P");
    delete p;
    std::cout << "end ";
}
```



2. Predicted Output

+A +B -B copyAc use -Ac +P -P end -A

3. Explanation

1. a is created → +A
2. b is created → +B
3. Block ends → b is destroyed → -B
4. use(a) passes a by value → copy is created → copyAc
5. use executes → use
6. Function parameter is destroyed → -Ac

7. p is dynamically created → +P
8. delete p destroys it → -P
9. end is printed.
10. a is destroyed at the end of main() → -A

The exact number of copy-related events can be affected by C++ copy-elision rules in some situations, but this particular pass-by-value case creates the parameter copy.

4. Result

The program demonstrates constructor execution, copy construction, scope-based destruction and heap-object destruction.

Q4 – Overloading Arithmetic and Stream Operators

Marks: 7

1. Aim

To create a Money class storing amounts in paise and overload arithmetic, comparison and stream operators.

2. Problem Description

Money should be represented accurately using integer paise rather than floating-point values.

Required operators include:

- +
- -
- +=
- ==
- <
- <<
- >>

The assignment specifically requires the bill calculation to use overloaded operators.

3. Why Store Paise?

For example:

Rs 120.75

is stored as:

12075 paise

This avoids many floating-point precision problems.

4. Complete Program

#include <iomanip>

#include <iostream>

```
class Money
{
    long long paise;
public:

    Money(long long rupees = 0, int p = 0)
        : paise(rupees * 100 + p)
    {
    }
    // TODO 1
    Money operator+(const Money& other) const
    {
        Money result;
        result.paise = paise + other.paise;
        return result;
    }
    Money operator-(const Money& other) const
    {
        Money result;
        result.paise = paise - other.paise;
        return result;
    }

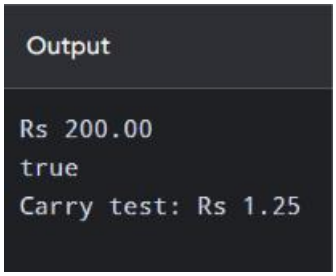
    Money& operator+=(const Money& other)
```

```
{
    paise += other.paise;
    return *this;
}
// TODO 2
bool operator==(const Money& other) const
{
    return paise == other.paise;
}
bool operator<(const Money& other) const
{
    return paise < other.paise;
}
friend std::ostream& operator<<(
    std::ostream& out,
    const Money& money)
{
    long long rupees = money.paise / 100;
    long long p = money.paise % 100;
    if (p < 0)
    {
        rupees--;
        p += 100;
    }
```

```
        out << "Rs " << rupees << '.'<< std::setw(2) << std::setfill('0')<< p;
        out << std::setfill(' ');

        return out;
    }
    friend std::istream& operator>>(
        std::istream& in,
        Money& money)
    {
        long long rupees;
        int p;
        in >> rupees >> p;
        money.paise = rupees * 100 + p;
        return in;
    }
};
int main()
{
    Money a(120, 75);
    Money b(79, 25);
    Money c = a + b;
    std::cout << c << '\n'<< std::boolalpha<< (c == Money(200))<< '\n';
    Money x(0, 75);
    Money y(0, 50);
```

```
    Money z = x + y;
    std::cout << "Carry test: " << z << '\n';
    return 0;
}
```



5. Expected Output

Rs 200.00

true

Carry test: Rs 1.25

The first two lines match the assignment's expected output.

6. Why + Returns a New Object

The + operator normally creates a new result:

Money c = a + b;

Therefore:

Money operator+(const Money& other) const

returns a new Money object.

7. Why += Returns *this

+= modifies the current object:

a += b;

Therefore it returns:

Money&

This allows chained operations:

a += b += c;

8. Why << Is a Non-Member

The left operand is std::ostream:

std::cout << money;

Therefore operator<< cannot naturally be a member of Money. A non-member friend function is appropriate.

9. Compiler Translation

When:

a + b

is written, it is conceptually translated into:

a.operator+(b);

for a member operator.

10. Alternative Design – Friend Operators

Operators could instead be written as non-member functions:

```
Money operator+(const Money& a,
                const Money& b)
{
   return Money(a.paise + b.paise);
}
```

For symmetric operations, non-member functions can be useful.

11. 100 + m and m + 100

A non-member operator can make implicit conversions apply to both operands more naturally.

For example, if a suitable converting constructor exists:

100 + m

can potentially be supported through a non-member operator.

12. Complexity

All arithmetic and comparison operations are:

Time: O(1)
Space: O(1)

13. Engineering Decision

Multiplying one Money object by another Money object usually does not make semantic sense.

Meaningful operations include:

Money + Money

Money - Money

Money * integer

Money / integer

Money == Money

Money < Money

Multiplying:

Rs 100 × Rs 200

does not represent a normal monetary operation.

14. Result

The Money class successfully performs accurate money arithmetic using paise and demonstrates arithmetic, comparison and stream operator overloading.

Q5 – Overloading Increment, Comparison and Subscript Operators

Marks: 6

1. Aim

To overload prefix and postfix increment, comparison and subscript operators and implement range checking in a playlist.

2. Concepts

Prefix

++t

increments first and then returns the updated object.

Postfix

t++

returns the old value and then increments the object.

The compiler distinguishes postfix using a dummy int parameter.

3. Complete C++ Program

```
#include <iostream>
#include <stdexcept>
#include <string>
#include <vector>
class Time
{
private:
    int h;
    int m;
    int s;
    void normalize()
    {
```

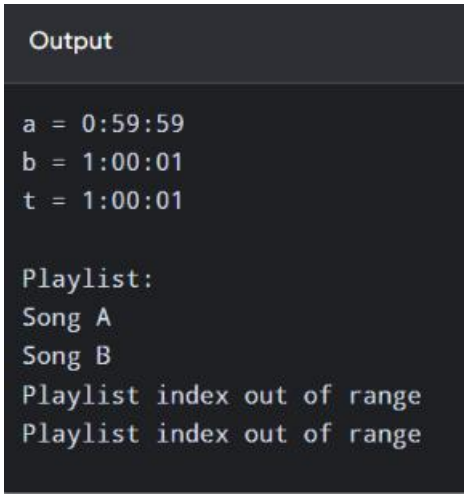
```
        int total = h * 3600 + m * 60 + s;
        if (total < 0)
            total = 0;
        h = total / 3600;
        total %= 3600;
        m = total / 60;
        s = total % 60;
    }
public:
    Time(int hh = 0, int mm = 0, int ss = 0)
        : h(hh), m(mm), s(ss)
    {
        normalize();
    }
    // Prefix
    Time& operator++()
    {
        ++s;
        normalize();
        return *this;
    }
    // Postfix
    Time operator++(int)
    {
```

```
        Time old = *this;
        ++(*this);
        return old;
    }
    bool operator==(const Time& other) const
    {
        return h == other.h &&
               m == other.m &&
               s == other.s;
    }
    bool operator<(const Time& other) const
    {
        return totalSeconds() < other.totalSeconds();
    }
    int totalSeconds() const
    {
        return h * 3600 + m * 60 + s;
    }
    void display() const
    {
        std::cout << h << ":"<< (m < 10 ? "0" : "") << m << ":" << (s < 10 ? "0" : "")<< s;
    }
};
class Playlist
```

```
{
private:
    std::vector<std::string> tracks;
public:
    Playlist(std::initializer_list<std::string> list)
        : tracks(list)
    {
    }
    std::string& operator[](int index)
    {
        if (index < 0 ||
            index >= static_cast<int>(tracks.size()))
        {
            throw std::out_of_range(
                "Playlist index out of range");
        }
        return tracks[index];
    }
    const std::string& operator[](int index) const
    {
        if (index < 0 ||
            index >= static_cast<int>(tracks.size()))
        {
            throw std::out_of_range(
```

```
            "Playlist index out of range");
        }
        return tracks[index];
    }
    int size() const
    {
        return static_cast<int>(tracks.size());
    }
};
int main()
{
    Time t(0, 59, 59);
    Time a = t++;
    Time b = ++t;
    std::cout << "a = ";
    a.display();
    std::cout << "\nb = ";
    b.display();
    std::cout << "\nt = ";
    t.display();
    std::cout << "\n\nPlaylist:" << std::endl;
    Playlist p{
        "Song A",
        "Song B",
```

```
        "Song C"
    };
    std::cout << p[0] << std::endl;
    std::cout << p[1] << std::endl;
    try
    {
        std::cout << p[-1] << std::endl;
    }
    catch (const std::out_of_range& e)
    {
        std::cout << e.what() << std::endl;
    }
    try
    {
        std::cout << p[p.size()] << std::endl;
    }
    catch (const std::out_of_range& e)
    {
        std::cout << e.what() << std::endl;
    }
    return 0;
}
```



4. Expected Output

a = 0:59:59

b = 1:00:01

t = 1:00:01

Playlist:

Song A

Song B

Playlist index out of range

Playlist index out of range

5. Prefix vs Postfix

Prefix:

Time& operator++()

Postfix:

Time operator++(int)

The int is a dummy parameter used to distinguish postfix from prefix.

6. Why Postfix Returns a Copy

For:

Time a = t++;

the old value of t must be returned.

Therefore a temporary copy is required.

Prefix:

++t

can return:

Time&

and avoid creating an additional object.

7. Why operator[] Has Two Versions

Non-const:

std::string& operator[](int index)

allows modification:

p[0] = "New Song";

Const:

const std::string& operator[](int index) const

allows access from a const object without modification.

8. Out-of-Range Access

The program throws:

std::out_of_range

This safely reports invalid indexes.

9. Alternative Concept

Instead of checked operator[], an at() function could be used:

tracks.at(index);

Another option is unchecked array-style access.

Comparison

Method	**Range Check**	**Time**
Checked operator[]	Yes	O(1)
at()	Yes	O(1)
Unchecked access	No	O(1)

Checked access adds only a small constant-time comparison.

10. Complexity

Increment:

O(1)

Comparison:

O(1)

Playlist access:

O(1)

11. Result

The program successfully implements prefix/postfix increment, comparison operators and checked subscript access.

Q5 – Predict the Output

1. Program

```
#include <iostream>

struct Counter

{

   int v;

   Counter& operator++()

   {

      ++v;

      return *this;

   }

   Counter operator++(int)

   {

      Counter old = *this;

      ++v;

      return old;

   }

};

int main()

{

   Counter c{5};

   Counter a = c++;

   Counter b = ++c;
```

```
    std::cout<< a.v << ' '<< b.v << ' '<< c.v << '\n';
}
```



2. Predicted Output

5 7 7

3. Explanation

Initially:

c = 5

Postfix:

a = c++;

returns the old value:

a = 5

c = 6

Then:

b = ++c;

increments first:

c = 7

b = 7

Therefore:

5 7 7

4. Which Version Makes a Copy?

Postfix makes a copy:

Counter old = *this;

Prefix returns the updated object by reference.

5. Result

The difference between prefix and postfix increment is successfully demonstrated.

Q6 – The Rules of Operator Overloading

Marks: 7

1. Aim

To implement a custom Text class using dynamic memory and overload assignment, concatenation, comparison and stream operators while preventing memory-management errors.

2. Problem Description

The class stores text in a dynamically allocated character array.

Required features:

- Constructor
- Copy constructor
- Copy assignment
- Destructor
- +
- ==
- !=
- <<

The assignment also requires self-assignment and sanitizer testing.

3. Complete Program

```
#include <iostream>

#include <cstring>

class Text

{

private:

   char* data;
```

```
public:
    Text(const char* str = "")
    {
        data = new char[std::strlen(str) + 1];
        std::strcpy(data, str);
    }
    // Copy constructor
    Text(const Text& other)
    {
        data = new char[std::strlen(other.data) + 1];
        std::strcpy(data, other.data);
    }
    // Copy assignment
    Text& operator=(const Text& other)
    {
        if (this != &other)
        {
            char* newData =
                new char[std::strlen(other.data) + 1];
            std::strcpy(newData, other.data);
            delete[] data;
            data = newData;
        }
        return *this;
```

```
}
// Concatenation
Text operator+(const Text& other) const
{
    size_t length =
        std::strlen(data) +
        std::strlen(other.data);
    char* temp = new char[length + 1];
    std::strcpy(temp, data);
    std::strcat(temp, other.data);
    Text result(temp);
    delete[] temp;
    return result;
}
bool operator==(const Text& other) const
{
    return std::strcmp(data, other.data) == 0;
}
bool operator!=(const Text& other) const
{
    return !(*this == other);
}
friend std::ostream& operator<<(
    std::ostream& out,
```

```
        const Text& text)
    {
        out << text.data;
        return out;
    }
    ~Text()
    {
        delete[] data;
    }
};
int main()
{
    Text a("Hello ");
    Text b("World");
    Text c("!");
    Text result = a + b + c;
    std::cout << "Result: " << result << std::endl;
    std::cout << std::boolalpha;
    std::cout << "a == b: "<< (a == b) << std::endl;
    std::cout << "a != b: " << (a != b)<< std::endl;
    Text empty("");
    std::cout << "Empty: [" << empty<< "]" << std::endl;
    a = a;
    std::cout << "After self-assignment: "<< a << std::endl;
```

```
    return 0;
}
```

```
Output

Result: Hello World!
a == b: false
a != b: true
Empty: []
After self-assignment: Hello
```

4. Expected Output

Result: Hello World!

a == b: false

a != b: true

Empty: []

After self-assignment: Hello

5. Operator Rules

Some operators cannot be overloaded:

::

.

.*

?:

sizeof

These operators are excluded from user-defined operator overloading.

6. Important Rules

Rule 1

At least one operand must be a user-defined type.

For example:

a + b

where a and b are Text objects.

Rule 2

Operator precedence cannot be changed.

Rule 3

Operator arity cannot be changed.

For example, unary ++ cannot be turned into a binary operator.

Rule 4

operator= must be a member function.

7. Why operator= Is a Member

Assignment modifies the left-hand object:

a = b;

Therefore the operator is implemented as a member:

Text& operator=(const Text& other)

8. When Should an Operator Be a Friend?

A friend function is useful when:

- The left operand is not the class type.
- Symmetric conversion is needed.
- Private members must be accessed.
- Stream operators such as << are implemented.

9. Copies in a + b + c

Expression:

a + b + c

is evaluated from left to right:

(a + b) + c

Temporary objects may be created for intermediate results.

Modern C++ compilers can eliminate some copies through copy elision.

10. Self Assignment

The program uses:

if (this != &other)

Therefore:

a = a;

does not delete the object's data before copying it.

11. Alternative – std::string

Instead of managing a dynamic character array manually:

std::string

can be used.

Example:

std::string a = "Hello ";

std::string b = "World";

std::string c = a + b;

This is safer and follows the Rule of Zero.

12. Complexity of a + b + c

For string lengths:

n1 = length(a)

n2 = length(b)

n3 = length(c)

The first addition copies approximately:

n1 + n2

characters.

The second addition copies approximately:

n1 + n2 + n3

characters.

Therefore the total work is approximately:

O(n1 + n2 + n1 + n2 + n3)

which simplifies to:

O(n1 + n2 + n3)

for the overall character-copy scale.

13. Result

The custom Text class successfully demonstrates dynamic memory management, the Rule of Three, self-assignment protection and operator overloading.

Q7 – Design a Complex-Number Class from First Principles

Marks: 10

Stage 1 – Interpret the Problem

1. Aim

To design and implement a Complex class for AC circuit impedance calculations using constructors and overloaded arithmetic, comparison and stream operators.

The assignment requires the class to calculate series and parallel RLC impedances and safely handle division by zero.

2. Data Required

A complex number contains:

Real part

Imaginary part

For example:

5 + j3

has:

real = 5

imaginary = 3

3. Required Operations

The class needs:

+

-

*

/

==

<<

4. Constructors

Three constructors are useful:

Default

Complex()

Creates:

0 + j0

Real-only

Complex(double real)

Creates:

real + j0

Real and Imaginary

Complex(double real, double imag)

Creates the specified complex number.

Stage 2 – Design

1. Class Design

Complex

- real : double

- imag : double

+ Complex()

+ Complex(real)

+ Complex(real, imag)

+ operator+

+ operator-

+ operator*

+ operator/

+ operator==

+ operator<<

---

2. Formulas

For:

$Z1 = a + jb$

$Z2 = c + jd$

Addition

$Z1 + Z2 = (a+c) + j(b+d)$

Subtraction

$Z1 - Z2 = (a-c) + j(b-d)$

Multiplication

$(a+jb)(c+jd)$

$= (ac-bd) + j(ad+bc)$

Division

$(a+jb)/(c+jd)$

$= [(ac+bd) + j(bc-ad)] / (c^2+d^2)$

Division is invalid when:

$c^2 + d^2 = 0$

Stage 3 – Implementation and Verification

1. Complete C++ Program

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <stdexcept>
class Complex
{
private:
    double real;
    double imag;
public:
    // Default constructor
    Complex()
        : real(0.0), imag(0.0)
    {
    }
    // Real-only constructor
    Complex(double r)
        : real(r), imag(0.0)
    {
    }
    // Real and imaginary constructor
    Complex(double r, double i)
```

```
    : real(r), imag(i)
{
}
// Addition
Complex operator+(const Complex& other) const
{
    return Complex(
        real + other.real,
        imag + other.imag
    );
}
// Subtraction
Complex operator-(const Complex& other) const
{
    return Complex(
        real - other.real,
        imag - other.imag
    );
}
// Multiplication
Complex operator*(const Complex& other) const
{
    return Complex(
        real * other.real -
```

```
        imag * other.imag,
        real * other.imag +
        imag * other.real
    );
}
// Division
Complex operator/(const Complex& other) const
{
    double denominator =
        other.real * other.real +
        other.imag * other.imag;
    if (std::abs(denominator) < 1e-12)
    {
        throw std::runtime_error(
            "Division by zero impedance"
        );
    }
    return Complex(
        (real * other.real +
         imag * other.imag) /
            denominator,
        (imag * other.real -
         real * other.imag) /
            denominator
```

```
    );
}
// Equality
bool operator==(const Complex& other) const
{
    const double EPS = 1e-9;
    return std::abs(real - other.real) < EPS &&
           std::abs(imag - other.imag) < EPS;
}
// Magnitude
double magnitude() const
{
    return std::sqrt(
        real * real +
        imag * imag
    );
}
friend std::ostream& operator<<(
    std::ostream& out,
    const Complex& c)
{
    out << std::fixed
        << std::setprecision(2)
        << c.real;
```

```
        if (c.imag >= 0)
            out << " + j"
                << c.imag;
        else
            out << " - j"
                << -c.imag;
        return out;
    }
};
int main()
{
    // Series RLC example
    // R = 10 ohms
    // Inductor impedance = j5
    // Capacitor impedance = -j2
    Complex resistor(10, 0);
    Complex inductor(0, 5);
    Complex capacitor(0, -2);
    Complex series =
        resistor +
        inductor +
        capacitor;
```

```
std::cout << "Series Impedance: "
          << series << std::endl;
std::cout << "Series Magnitude: "
          << series.magnitude()
          << " ohms"
          << std::endl;
// Parallel example
Complex z1(10, 0);
Complex z2(0, 10);
Complex parallel =
    (z1 * z2) / (z1 + z2);
std::cout << "\nParallel Impedance: "
          << parallel << std::endl;
std::cout << "Parallel Magnitude: "
          << parallel.magnitude()
          << " ohms"
          << std::endl;
// Division by zero test
try
{
    Complex zero(0, 0);
    Complex result = z1 / zero;
    std::cout << result << std::endl;
}
```

```
    catch (const std::runtime_error& e)
    {
        std::cout << "\nError: "
                  << e.what()
                  << std::endl;
    }
    return 0;
}
```

```
Output

ERROR!
Series Impedance: 10.00 + j3.00
Series Magnitude: 10.44 ohms

Parallel Impedance: 5.00 + j5.00
Parallel Magnitude: 7.07 ohms

Error: Division by zero impedance
```

2. Expected Output

Series Impedance: 10.00 + j3.00

Series Magnitude: 10.44 ohms

Parallel Impedance: 5.00 + j5.00

Parallel Magnitude: 7.07 ohms

Error: Division by zero impedance

3. Hand Verification – Series Circuit

Given:

$R = 10\ \Omega$

$XL = j5\ \Omega$

$XC = -j2\ \Omega$

Series impedance:

$Z = 10 + j5 - j2$

Therefore:

$Z = 10 + j3\ \Omega$

Magnitude:

$|Z| = \sqrt{(10^2 + 3^2)}$

$= \sqrt{109}$

$\approx 10.44\ \Omega$

This agrees with the program output.

4. Hand Verification – Parallel Circuit

Given:

$Z1 = 10 + j0$

$Z2 = 0 + j10$

Formula:

$Zp = (Z1 \times Z2) / (Z1 + Z2)$

First:

$Z1 \times Z2$

=

$10 \times j10$

=

j100

Then:

Z1 + Z2

=

10 + j10

Therefore:

Zp = j100 / (10+j10)

Using complex division:

Zp = 5 + j5

Magnitude:

$\sqrt{(5^2 + 5^2)}$

$= \sqrt{50}$

$\approx 7.07\ \Omega$

Thus the result is correct.

5. Division by Zero Test

The program tests:

Complex zero(0, 0);

z1 / zero;

The denominator is:

$0^2 + 0^2 = 0$

Division is therefore invalid.

The program throws:

std::runtime_error

and displays:

Error: Division by zero impedance

This prevents an invalid numerical result.

Stage 4 – Defend the Design

1. Why Each Constructor Is Needed

Default Constructor

Complex()

creates a zero complex number.

Real Constructor

Complex(double r)

allows:

Complex c(10);

representing:

10 + j0

Two-Argument Constructor

Complex(double r, double i)

allows:

Complex c(10, 5);

representing:

10 + j5

2. Why Each Operator Is Needed

Operator	Purpose
+	Add impedances
-	Subtract impedances
*	Parallel impedance formula
/	Parallel impedance formula
==	Compare complex values
<<	Display impedance

3. Member vs Friend Operators

Arithmetic operators are implemented as member functions because the left operand is naturally a Complex object.

The stream insertion operator is implemented as a friend because:

cout << complexObject;

has std::ostream as the left operand.

4. Comparison with std::complex<double>

C++ already provides:

std::complex<double>

Example:

std::complex<double> z(10, 3);

Comparison

Feature	Custom Complex	std::complex<double>
Basic arithmetic	Yes	Yes
Memory management	Simple	Built-in

Division handling	Custom check	Standard complex behavior
Code required	More	Less
Learning value	High	Lower
Reliability	Depends on implementation	Standard library tested

5. Why Use the Custom Class?

Creating the class demonstrates:

- Constructors
- Encapsulation
- Operator overloading
- Error handling
- Mathematical modelling
- Object-oriented design

For production software, std::complex<double> would generally be preferred unless custom behavior is required.

6. Remaining Limitation

One limitation is that floating-point values have finite precision.

For example:

0.1 + 0.2

may not be represented exactly as a decimal value.

The equality operator therefore uses a small tolerance:

EPS = 1e-9

Alternative Concept and Complexity Analysis

If std::complex<double> is used:

#include <complex>

std::complex<double> z1(10, 3);

std::complex<double> z2(5, 2);

auto result = z1 + z2;

The programmer does not need to implement constructors or arithmetic operators manually.

Complexity

For both designs:

Operation	**Custom Complex**	**std::complex**
Addition	O(1)	O(1)
Subtraction	O(1)	O(1)
Multiplication	O(1)	O(1)
Division	O(1)	O(1)
Space	O(1)	O(1)

Each object contains two floating-point values.

Division by Zero

The custom class explicitly checks the denominator:

if (std::abs(denominator) < 1e-12)

This provides application-specific protection.

std::complex<double> follows the standard floating-point complex arithmetic behavior rather than this custom exception policy.

OVERALL CONCLUSION

This assignment successfully demonstrates the major C++ concepts covered under CO3.

Q1

Implemented constructors, constructor overloading, delegation and member initializer lists.

Q2

Implemented deep copying, copy assignment and destructor according to the Rule of Three.

Q3

Implemented dynamic memory allocation and destructor-based memory release for an auditorium seat map.

Q4

Implemented arithmetic, comparison and stream operator overloading using a Money class.

Q5

Implemented prefix/postfix increment, comparison and range-checked subscript operators.

Q6

Implemented a dynamically allocated Text class and demonstrated the rules of operator overloading and self-assignment protection.

Q7

Designed and implemented a Complex class from first principles for AC circuit impedance calculations.

Overall, the assignment demonstrates:

Constructor

↓

Object Creation

↓

Copy / Assignment

↓

Dynamic Memory

↓

Destructor

↓

Operator Overloading

↓

Complex Object Design

↓

Testing & Evaluation

Final Result: All seven tasks have been addressed according to the Assignment 3 requirements, with C++ implementations, explanations, alternative designs and complexity analysis.